# ESC180: Introduction to Computer Programming

# Exam Review

Brendan Tang

Fall 2026

## Contents

**1 Python Fundamentals** **2**
1.1 Comments and Output . . . 2
1.2 Variables and Data Types . . . 2
1.2.1 Interlude (Local and Global Variables) . . . 3
1.3 Booleans and Operators . . . 3
1.4 Conditional Statements . . . 4
1.5 Loops . . . 4
1.6 Strings . . . 5
1.7 Functions . . . 6
1.8 User Input . . . 6
1.9 Lists and Tuples . . . 7
1.10 Dictionaries . . . 7

# 1 Python Fundamentals

## 1.1 Comments and Output

We can use the `print()` function to display text or output values. Text must be inside quotes, otherwise Python will return a syntax error.

**Example.**

```
print("Hello world!")
print("I love Praxis!") # Automatically prints on a new line
```

*Output.*

```
Hello world!
I love Praxis!
```

Any text after `#` is a comment. These will not be read by Python, and are used to make code more readable. You can also do math inside of a print statement. Observe the absence of quotes.

**Example.**

```
print("What is 3 + 10?")
print("3 + 10 =", 3 + 10) # Prints 13, as 3 + 10 = 13
```

*Output.*

```
What is 3 + 10?
3 + 10 = 13
```

## 1.2 Variables and Data Types

Here are some commonly used data types in Python.

| Data Type | Example |
|---|---|
| str | `"hi!"` |
| int | `120` |
| float | `12.5` |
| bool | `True` |
| list | `["an", "ban", "can"]` |
| tuple | `("an", "ban", "can")` |
| dict | `{"ESC101": 70, "ESC194": 100}` |

Variables are used to store data values. A variable is created when you assign a value to it with the assignment operator, =.

**Example.**

```
toggled = False
name = "John Doe"

x1 = 50 # stores 50
x2 = float(50) # stores 50.0

y1 = 11.1 # stores 11.1
y2 = int(11.1) # stores 11
```

Observe how Python automatically determines the type of a variable, unless you explicitly cast a different data type.

### 1.2.1 Interlude (Local and Global Variables)

Variables that are created outside of a function are known as global variables. Global variables can be used by everyone, both inside of functions and outside. Variables that are created inside a function are known as local variables. Local variables can only be used inside their functions.

We can however create global variables inside a function by using the `global` keyword.

## 1.3 Booleans and Operators

Comparing two values generates a Boolean value, `True` or `False`. You can also use `bool()`, which will return `True` unless you pass in numeric zeros, empty sequences/collections, or `None` in which case `False` is returned.

Here are some arithmetic operators.

| Arithmetic Operator | Name |
|---|---|
| + | Addition |
| - | Subtraction |
| * | Multiplication |
| / | Division |
| % | Modulus |
| ** | Exponentiation |
| // | Floor division |

Here are some assignment operators.

| Assignment Operator | Example | Equivalent To |
|---|---|---|
| = | `x = 10` | `x = 10` |
| += | `x += 10` | `x = x + 10` |
| -= | `x -= 10` | `x = x - 10` |
| *= | `x *= 10` | `x = x * 10` |
| /= | `x /= 10` | `x = x / 10` |
| %= | `x %= 10` | `x = x % 10` |
| **= | `x **= 10` | `x = x ** 10` |
| //= | `x //= 10` | `x = x // 10` |

Here are some comparison operators.

| Comparison Operator | Name |
|---|---|
| == | Equal |
| != | Not equal |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal to |
| <= | Less than or equal to |

.

Here are some logical, identity, and membership operators.

| Operator | Description |
|---|---|
| `and` | Returns `True` if both statements are `True` |
| `or` | Returns `True` if one of the statements is `True` |
| `not` | Reverses the result, returns `False` if the result is `True` |
| `is` | Returns `True` if both variables are the same object |
| `is not` | Returns `True` if both variables are not the same object |
| `in` | Returns `True` if a sequence with the specified value is present in the object |
| `not in` | Returns `True` if a sequence with the specified value is not present in the object |

## 1.4 Conditional Statements

The `if` statement evaluates an expression to determine its boolean value. If `True`, the code block inside the statement is executed. If `False`, the code block is skipped.

**Example.**

```
if grade >= 85:
    print("GPV: 4.0")
elif grade >= 80: # Python will move on to this if the original is False
    print("GPV: 3.7")
elif grade >= 77: # Same for this, if both of the above are False
    print("GPV: 3.3")
else: # Code block will be executed if all of the above are False
    print("Is this Praxis?")
```

Notice that in Python, indentation is used to determine which statements belong together. If we have a lot of cases, we may wish to use a `match` statement instead.

**Example.**

```
match color:
    case "red":
        print("Roses are red.")
    case "yellow" | "green": # We can check for more than one value match
        print("Grass is green, yellow if dead.")
    case "blue": if time == "day" # We can add an extra condition check
        print("The sky is blue.")
    case _: # This will execute if there are no matches
        print("This color is boring.")
```

## 1.5 Loops

A `for` loop is used for iterating over a sequence (a list, a tuple, a set, or a string). Most commonly, we use the `range` data type, which represents an immutable sequence of numbers.

**Example.**

```
for i in range(3): # Notice how by default, we begin at 0
    print("I am", i)

print(" ")

for i in range(3, 10, 3): # Syntax is (start, stop, step)
    print("I am", i)
```

*Output.*

```
I am 0
I am 1
I am 2

I am 3
I am 6
I am 9
```

A `while` loop will execute a set of statements as long as a condition is `True`. If the condition is always `True`, we could end up in an infinite loop.

**Example.**

```
height = 1
while True: # This loop will continue forever, unless...
    print("I am", height, "feet tall!")
    height += 1
    if height > 6:
        break # This will exit the loop
```

*Output.*

```
I am 1 feet tall!
I am 2 feet tall!
I am 3 feet tall!
I am 4 feet tall!
I am 5 feet tall!
I am 6 feet tall!
```

Observe the use of `break` above. If we wish to stop the current iteration but stay in the loop, we can use `continue`. We can also use an `else` statement to run a block of code when the condition is no longer `True`. This means that exiting the loop using `break` will not run the code inside the `else` statement.

## 1.6 Strings

Strings in Python are arrays of unicode characters. However, since Python doesn't have a character data type, a character is simply a string with length 1.

Here are some common string methods. Note that there are many others, these are just the most common.

| Syntax | Description |
|---|---|
| `s[i]` | Get the character at position `i` |
| `len(s)` | Returns the length of `s` |
| `str0 in s` | Checks if `str0` is in `s` |
| `s[start:stop:step]` | Extracts a portion of `s` |
| `s.upper()` | Returns `s` in upper case |
| `s.lower()` | Returns `s` in lower case |
| `s.strip()` | Returns `s` without leading/trailing whitespace |
| `s.replace(str1, str2)` | Returns `s`, replacing `str1` with `str2` |
| `s.split(char)` | Returns a list of substrings, with separator `char` |

Python allows negative indexing. Also, since strings are arrays, we can loop through the characters in a string with a `for` loop.

**Example.**

```
i = 1
for char in "mathematics":
    print((char * i) + (" " * 2 * (11 - i)) + (char * i))
    i += 1
```

*Output.*

```
m                    m
aa                  aa
ttt                ttt
hhhh              hhhh
eeeee            eeeee
mmmmmm          mmmmmm
aaaaaaa        aaaaaaa
tttttttt      tttttttt
iiiiiiiii    iiiiiiiii
ccccccccc  ccccccccc
sssssssssssssssssss
```

## 1.7 Functions

A function is defined using the `def` keyword. To call a function, write its name followed by parentheses.

**Example.**

```
def print_hello():
    print("Hello there!")

print_hello()
```

*Output.*

```
Hello there!
```

Functions can send data back to the code that called them using the `return` statement. When a function reaches a `return` statement, it stops executing and sends the result back. If a function doesn't have a `return` statement, it returns `None` by default.

We can also pass information into functions as arguments. These are specified after the function name, inside the parentheses. You can also assign default values to parameters.

**Example.**

```
def greeting_message(name1, name2 = "alien"): # name2 has a default value
    return "Hi " + name1 + ", hello " + name2 + "!"

greeting_message("cat", "dog") # Nothing is printed (why?)
print(greeting_message("cow")) # Here we exclude a second argument
```

*Output.*

```
Hi cow, hello alien!
```

If you do not know how many arguments will be passed into your function, add a $*$ before the parameter name to send a tuple of arguments. You can also add a $**$ before the parameter name to send the function a dictionary of arguments.

## 1.8 User Input

You can use `input()` to ask the user for input. You can include a prompt inside the parentheses, which is a message that will display before the user input on the same line. The input from the user is always treated as a string.

## 1.9 Lists and Tuples

Lists are used to store multiple items in a single variable. List items are indexed, the first item has `[0]`, the second item has index `[1]`, etc. A list can contain different data types.

Here are some common list methods.

| Syntax | Description |
|---|---|
| `l[i]` | Get the item at position `i` in `l` |
| `len(l)` | Returns the length of `l` |
| `l[start:stop:step]` | Extracts a portion of `l` |
| `l.insert(i, item)` | Inserts `item` in `l` at `i` |
| `l.append(item)` | Appends `item` to `l` |
| `list1.extend(list2)` | Appends `list1` to `list2` |
| `l.remove(item)` | Removes the first occurence of `item` from `l` |
| `l.pop(i)` | Removes `i` from `l` |
| `l.clear()` | Empties `l` |
| `l.sort()` | Sorts `l` in ascending order |
| `l.sort(reverse = True)` | Sorts `l` in descending order |
| `l.reverse()` | Reverses the order of `l` |
| `l.copy()` | Copies `l` |

## 1.10 Dictionaries

Dictionaries are used to score data values in key:value pairs. I don't feel like typing anymore so here is an example.

**Example.**

```
grades = {
    "Praxis": 60,
    "Calculus": 100,
    "Civ": 80
}
print("My grade in Praxis is " + grades.get("Praxis"))

if "Civ" in grades: # Checks if "Civ" is a key in grades
    print("I am taking CIV102 this term!")
else:
    print("I am not taking CIV102 this term.")

grades.pop("Civ") # Removes the pair with "Civ"

if "Civ" in grades: # Checks if "Civ" is a key in grades
    print("I am taking CIV102 this term!")
else:
    print("I am not taking CIV102 this term.")

grades.clear() # Empties the dictionary
```

*Output.*

```
My grade in Praxis is 60
I am taking CIV102 this term!
I am not taking CIV102 this term.
```